

UNIVERSIDAD PERUANA LOS ANDES
FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA DE
SISTEMAS Y COMPUTACIÓN



CURSO: Base de Datos II

DOCENTE: Ing. Fernandez Bejarano Raul Enrique

ESTUDIANTE: Nuñez Michel Jair

CICLO: V

SECCION: B1

TEMA 1. ANÁLISIS DE RENDIMIENTO CON SQL PROFILER Y EXTENDED EVENTS

1. Proyecto 1: Monitoreo de Consultas Lentas con Extended Events

Enunciado

La Oficina de Tecnologías de Información de la universidad ha detectado que el módulo de registro de expedientes de bachiller y título profesional presenta lentitud durante las horas de mayor demanda.

Se solicita implementar una sesión de monitoreo utilizando **Extended Events** para capturar todas las consultas cuya duración sea superior a **1000 milisegundos** sobre la base de datos **GradosTitulos**.

El estudiante deberá identificar las consultas más costosas ejecutadas sobre los esquemas:

- Academico
- Tramite
- Evaluacion
- Documento

Al finalizar, deberá generar un reporte con las consultas capturadas.

2. Script de la solución en T-SQL

```
USE master;
GO

IF EXISTS
(
    SELECT *
    FROM sys.server_event_sessions
    WHERE name='XE_ConsultasLentas'
)
DROP EVENT SESSION XE_ConsultasLentas
ON SERVER;
GO

CREATE EVENT SESSION XE_ConsultasLentas
ON SERVER
ADD EVENT sqlserver.sql_statement_completed
(
    ACTION
    (
        sqlserver.sql_text,
        sqlserver.database_name,
        sqlserver.username
    )
    WHERE
    (
        duration>1000000
        AND sqlserver.database_name='GradosTitulos'
    )
)
ADD TARGET package0.event_file
(
    SET filename='C:\SQL2025\ExtendedEvents\XE_ConsultasLentas.xel'
);
GO

ALTER EVENT SESSION XE_ConsultasLentas
ON SERVER
STATE=START;
GO

USE GradosTitulos;
GO

WAITFOR DELAY '00:00:02';
SELECT * FROM Academico.Persona;
GO

WAITFOR DELAY '00:00:02';
SELECT * FROM Tramite.Tramite;
GO

WAITFOR DELAY '00:00:02';
SELECT * FROM Evaluacion.Sustentacion;
GO

WAITFOR DELAY '00:00:02';
SELECT * FROM Documento.Resolucion;
GO

SELECT
event_data.value('(event/@timestamp)[1]', 'datetime2') AS FechaHora,
event_data.value('(event/action[@name="database_name"]/value)[1]', 'nvarchar(100)') AS BaseDatos,
event_data.value('(event/action[@name="username"]/value)[1]', 'nvarchar(100)') AS Usuario,
event_data.value('(event/action[@name="sql_text"]/value)[1]', 'nvarchar(max)') AS Consulta,
event_data.value('(event/data[@name="duration"]/value)[1]', 'bigint')/1000 AS Duracion_ms
FROM
(
    SELECT
    CAST(event_data AS XML) AS event_data
    FROM sys.fn_xe_file_target_read_file
    (
        'C:\SQL2025\ExtendedEvents\XE_ConsultasLentas*.xel',
        NULL,
        NULL,
        NULL,
        0
    ) X
    WHERE
    event_data.value('(event/action[@name="sql_text"]/value)[1]', 'nvarchar(max)')
    LIKE '%Academico.%'
    OR
    event_data.value('(event/action[@name="sql_text"]/value)[1]', 'nvarchar(max)')
    LIKE '%Tramite.%'
    OR
    event_data.value('(event/action[@name="sql_text"]/value)[1]', 'nvarchar(max)')
    LIKE '%Evaluacion.%'
    OR
    event_data.value('(event/action[@name="sql_text"]/value)[1]', 'nvarchar(max)')
    LIKE '%Documento.%'
)
ORDER BY Duracion_ms DESC;
GO
```

Resultados

Mensajes

	FechaHora	BaseDatos	Usuario	Consulta	Duracion_ms
1	2026-07-12 03:22:41.4820000	GradosTitulos	JAIR\jair	WAITFOR DELAY '00:00:02'; SELECT * FROM Evaluaci...	2015
2	2026-07-12 03:22:43.5090000	GradosTitulos	JAIR\jair	WAITFOR DELAY '00:00:02'; SELECT * FROM Docume...	2015
3	2026-07-12 03:42:47.7960000	GradosTitulos	JAIR\jair	WAITFOR DELAY '00:00:02'; SELECT * FROM Documen...	2015
4	2026-07-12 15:19:10.3340000	GradosTitulos	JAIR\jair	WAITFOR DELAY '00:00:02'; SELECT * FROM Documen...	2015
5	2026-07-12 03:42:45.7680000	GradosTitulos	JAIR\jair	WAITFOR DELAY '00:00:02'; SELECT * FROM Evaluacio...	2014
6	2026-07-12 03:42:43.7440000	GradosTitulos	JAIR\jair	WAITFOR DELAY '00:00:02'; SELECT * FROM Tramite.T...	2013
7	2026-07-12 03:22:37.4420000	GradosTitulos	JAIR\jair	WAITFOR DELAY '00:00:02'; SELECT * FROM Academi...	2013
8	2026-07-12 14:34:59.1120000	GradosTitulos	JAIR\jair	WAITFOR DELAY '00:00:02'; SELECT * FROM Tramite...	2012
9	2026-07-12 03:00:21.3350000	GradosTitulos	JAIR\jair	WAITFOR DELAY '00:00:02'; SELECT * FROM Tramite...	2011

Consulta ejecutada correctamente.

localhost\SQLEXPRESS (17.0... JAIR\jair (70) GradosTitulos 00:00:00 Fila: 1, Columna: 1 25 filas

Consulta ejecutada correctamente.

localhost\SQLEXPRESS (17.0... | JAIR\jair (70) | GradosTitulos | 00:00:00 | Fila: 1, Columns: 1 | 25 filas

3. Justificación técnica de la solución aplicada

Se implementó una sesión de Extended Events para capturar consultas cuya duración supera los 1000 milisegundos en la base de datos **GradosTitulos**, permitiendo identificar las operaciones de mayor costo ejecutadas sobre los diferentes esquemas.

4. Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizó Extended Events por su bajo impacto en el rendimiento del servidor y se almacenaron los eventos en un archivo .xel para facilitar su análisis posterior. Finalmente, se generó un reporte ordenado por tiempo de ejecución para identificar las consultas más lentas.

Proyecto 2: Implementación de Monitoreo Integral con Extended Events

1. Enunciado

Durante los procesos de matrícula para obtención del título profesional, varios usuarios reportan tiempos elevados en la generación de resoluciones.

La Dirección de Informática solicita construir una solución de monitoreo que automáticamente registre:

- Consultas lentas.
- Deadlocks.
- Bloqueos.
- Errores superiores al nivel 16.
- Uso excesivo de CPU.

Toda la información deberá almacenarse en un archivo de eventos para su posterior análisis.

2. Script de la solución en T-SQL

```
USE master;
GO

IF EXISTS
(
    SELECT *
    FROM sys.server_event_sessions
    WHERE name='XE_MonitoreoIntegral'
)
DROP EVENT SESSION XE_MonitoreoIntegral
ON SERVER;
GO

CREATE EVENT SESSION XE_MonitoreoIntegral
ON SERVER
(
    ADD EVENT sqlserver.sql_statement_completed
    (
        ACTION
        (
            sqlserver.sql_text,
            sqlserver.database_name,
            sqlserver.username
        )
        WHERE
        (
            duration>1000000
            AND sqlserver.database_name='GradosTitulos'
        )
    ),
    ADD EVENT sqlserver.rpc_completed
    (
        ACTION
        (
            sqlserver.sql_text,
            sqlserver.database_name,
            sqlserver.username
        )
        WHERE
        (
            cpu_time>1000
            AND sqlserver.database_name='GradosTitulos'
        )
    ),
    ADD EVENT sqlserver.error_reported
    (
        ACTION
        (
            sqlserver.sql_text,
            sqlserver.database_name,
            sqlserver.username
        )
        WHERE
        (
            severity>=16
        )
    ),
    ADD EVENT sqlserver.xl_deadlock_report,
    ADD EVENT sqlserver.lock_timeout_greater_than_0
    (
        ACTION
        (
            sqlserver.sql_text,
            sqlserver.database_name,
            sqlserver.username
        )
    ),
    ADD TARGET package0.event_file
    (
        SET filename='C:\SQL2025\ExtendedEvents\XE_MonitoreoIntegral*.xel'
    )
);
GO

ALTER EVENT SESSION XE_MonitoreoIntegral
ON SERVER
STATE=START;
GO

USE GradosTitulos;
GO

WAITFOR DELAY '00:00:02';
SELECT * FROM Academico.Persona;
GO

WAITFOR DELAY '00:00:02';
SELECT * FROM Tramite.Tranite;
GO

WAITFOR DELAY '00:00:02';
SELECT * FROM Evaluacion.Sustentacion;
GO

WAITFOR DELAY '00:00:02';
SELECT * FROM Documento.Resolucion;
GO

SELECT
    object_name AS TipoEvento,
    CAST(event_data AS XML) AS Evento
FROM sys.fn_xe_file_target_read_file
(
    'C:\SQL2025\ExtendedEvents\XE_MonitoreoIntegral*.xel',
    NULL,
    NULL,
    NULL,
    NULL
)
ORDER BY object_name;
GO
```

Resultados		Mensajes	
TipoEvento	Evento		
1	error_reported	event_name="error_reported" package="sqlserver"	
2	rpc_completed	event_name="rpc_completed" package="sqlserver"	
3	rpc_completed	event_name="rpc_completed" package="sqlserver"	
4	rpc_completed	event_name="rpc_completed" package="sqlserver"	
5	rpc_completed	event_name="rpc_completed" package="sqlserver"	
6	rpc_completed	event_name="rpc_completed" package="sqlserver"	
7	rpc_completed	event_name="rpc_completed" package="sqlserver"	
8	rpc_completed	event_name="rpc_completed" package="sqlserver"	

Consulta ejecutada correctamente.

localhost\SQLEXPRESS (17.0... | JAIRYair (70) | GradosTitulos | 00:00:00 | Fila: 1, Columna: 1 | 31 filas

3. Justificación técnica de la solución aplicada

Se implementó una sesión de **Extended Events** para monitorear automáticamente la base de datos **GradosTitulos**, registrando consultas lentas, deadlocks, bloqueos, errores críticos y alto consumo de CPU. Los eventos se almacenan en un archivo **.xel** para facilitar su análisis posterior.

4. Explicación de las buenas prácticas utilizadas

Se utilizó **Extended Events** por su bajo impacto en el rendimiento del servidor. Además, se configuró el monitoreo únicamente para la base de datos **GradosTitulos** y se almacenaron los eventos en archivos **.xel**, permitiendo un seguimiento y diagnóstico eficiente del rendimiento del sistema.

Este formato es **más corto, preciso y entendible**, ideal para un informe universitario y puedes mantenerlo como modelo para los siguientes proyectos.

Proyecto 3: Sistema Permanente de Diagnóstico del Rendimiento

1. Enunciado

La universidad desea implementar un sistema permanente de diagnóstico del rendimiento para la base de datos **GradosTitulos**.

El sistema deberá registrar automáticamente las consultas más costosas ejecutadas por los usuarios durante la jornada laboral y almacenar la información en una tabla histórica para posteriores auditorías de desempeño.

El estudiante deberá diseñar un mecanismo que permita:

- Iniciar la sesión de monitoreo.
- Registrar las consultas ejecutadas sobre la base de datos **GradosTitulos**.
- Importar periódicamente la información del archivo de eventos a una tabla de auditoría.
- Generar un reporte de las consultas con mayor duración, mayor consumo de CPU y mayor número de lecturas lógicas.

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

IF OBJECT_ID('dbo.AuditoriaConsultas','U') IS NOT NULL
DROP TABLE dbo.AuditoriaConsultas;
GO

CREATE TABLE dbo.AuditoriaConsultas
(
    Id INT IDENTITY(1,1) PRIMARY KEY,
    FechaHora DATETIME2,
    ConsultaSQL NVARCHAR(MAX),
    Duracion_ms BIGINT,
    CPU_ms BIGINT,
    LecturasLogicas BIGINT
);
GO

USE master;
GO

IF EXISTS
(
    SELECT *
    FROM sys.server_event_sessions
    WHERE name='XE_Diagnostico'
)
DROP EVENT SESSION XE_Diagnostico
ON SERVER;
GO

CREATE EVENT SESSION XE_Diagnostico
ON SERVER

ADD EVENT sqlserver.sql_statement_completed
(
    ACTION
    (
        sqlserver.sql_text,
        sqlserver.database_name
    )
    WHERE
    (
        sqlserver.database_name='GradosTitulos'
    )
)

ADD TARGET package0.event_file
(
    SET filename='C:\SQL2025\ExtendedEvents\XE_Diagnostico*.xel'
);
GO
```

```
ALTER EVENT SESSION XE_Diagnostico
ON SERVER
STATE=START;
GO

USE GradosTitulos;
GO

INSERT INTO dbo.AuditoriaConsultas
(
    FechaHora,
    ConsultaSQL,
    Duracion_ms,
    CPU_ms,
    LecturasLogicas
)
SELECT
    X.Evento.value('event/@timestamp')[1], 'datetime2'),
    X.Evento.value('event/action[@name="sql_text"]/@value')[1], 'nvarchar(max)'),
    X.Evento.value('event/data[@name="duration"]/@value')[1], 'bigint')/1000,
    X.Evento.value('event/data[@name="cpu_time"]/@value')[1], 'bigint'),
    X.Evento.value('event/data[@name="logical_reads"]/@value')[1], 'bigint')
FROM
(
    SELECT CAST(event_data AS XML) Evento
    FROM sys.fn_xe_file_target_read_file
    (
        'C:\SQL2025\ExtendedEvents\XE_Diagnostico*.xel',
        NULL,
        NULL,
        NULL
    ) X;
) X;
GO

SELECT TOP 10
    FechaHora,
    Duracion_ms,
    CPU_ms,
    LecturasLogicas,
    ConsultaSQL
FROM dbo.AuditoriaConsultas
ORDER BY Duracion_ms DESC;
GO

SELECT TOP 10
    FechaHora,
    CPU_ms,
    Duracion_ms,
    LecturasLogicas,
    ConsultaSQL
FROM dbo.AuditoriaConsultas
ORDER BY CPU_ms DESC;
GO

SELECT TOP 10
    FechaHora,
    LecturasLogicas,
    Duracion_ms,
    CPU_ms,
    ConsultaSQL
FROM dbo.AuditoriaConsultas
ORDER BY LecturasLogicas DESC;
GO
```

Resultados		Mensajes			
	FechaHora	Duracion_ms	CPU_ms	LecturasLogicas	ConsultaSQL
1	2026-07-12 15:19:10.3340000	2015	0	0	WAITFOR DELAY '00:00:02'; SELECT * FROM Documen...
2	2026-07-12 16:02:43.3310000	2015	0	0	WAITFOR DELAY '00:00:02'; SELECT * FROM Tramlite.T...
3	2026-07-12 16:02:43.3310000	2015	0	0	WAITFOR DELAY '00:00:02'; SELECT * FROM Tramlite.T...
4	2026-07-12 16:11:54.8800000	2014	0	0	WAITFOR DELAY '00:00:02'; SELECT * FROM Documen...
5	2026-07-12 16:11:54.8800000	2014	0	0	WAITFOR DELAY '00:00:02'; SELECT * FROM Documen...
6	2026-07-12 16:02:41.3060000	2011	0	0	WAITFOR DELAY '00:00:02'; SELECT * FROM Academic...
	FechaHora	CPU_ms	Duracion_ms	LecturasLogicas	ConsultaSQL
1	2026-07-12 15:19:49.0810000	46000	75	1485	EXEC dbo.sp_ImportarEventos@E;
2	2026-07-12 16:02:50.7930000	32000	51	10727	SELECT event_data.value[event/@timestamp][1]...
3	2026-07-12 16:02:50.7930000	32000	51	10727	SELECT event_data.value[event/@timestamp][1]...
4	2026-07-12 15:19:12.1280000	0	1	6	IF OBJECT_ID('dbo.sp_ImportarEventos@E') IS ...
5	2026-07-12 15:19:10.3430000	0	0	0	USE GradosTitulos;
6	2026-07-12 15:19:10.3340000	0	0	0	WAITFOR DELAY '00:00:02'; SELECT * FROM D...
	FechaHora	LecturasLogicas	Duracion_ms	CPU_ms	ConsultaSQL
1	2026-07-12 16:02:50.7930000	10727	51	32000	SELECT event_data.value[event/@timestamp][1]...
2	2026-07-12 16:02:50.7930000	10727	51	32000	SELECT event_data.value[event/@timestamp][1]...
3	2026-07-12 15:19:49.0810000	1485	75	46000	EXEC dbo.sp_ImportarEventos@E;
4	2026-07-12 15:46:28.1110000	489	4	0	IF OBJECT_ID('dbo.AuditoriaConsultas@E') IS NOT ...
5	2026-07-12 16:12:22.9060000	475	14	0	SELECT object_name AS TipoEvento, CAST(even...
6	2026-07-12 16:12:22.9060000	475	14	0	SELECT object_name AS TipoEvento, CAST(even...

Consulta ejecutada correctamente.

localhost\SQLEXPRESS (17.0.0.1) - JAIR(vair (70)) - GradosTitulos - 00:00:00 - Fila: 1, Columna: 1 - 30 Fila

Consulta ejecutada correctamente.

localhost\SQLEXPRESS (17.0 ... JAIRYair (70) GradosTitulos 00:00:00 Fila: 1, Columna: 1 30 filas

3. Justificación técnica

Se implementó una sesión de **Extended Events** para registrar las consultas ejecutadas en la base de datos **GradosTitulos** y almacenar la información en una tabla histórica de auditoría, permitiendo analizar el rendimiento y realizar auditorías de desempeño.

4. Explicación de las buenas prácticas

Se utilizó **Extended Events** por su bajo impacto en el servidor y una tabla de auditoría para conservar un historial de consultas. Además, se generaron reportes que permiten identificar rápidamente las consultas con mayor duración, consumo de CPU y lecturas lógicas.

TEMA 2. ESTADÍSTICAS E ÍNDICES

1. Proyecto 1: Optimización de Índices para Consultas de Estudiantes

Enunciado

Las consultas realizadas sobre las tablas **Academico.Persona**, **Academico.Matricula** y **Tramite.Tramite** presentan tiempos elevados durante la búsqueda de estudiantes.

Se solicita desarrollar un conjunto de scripts que permitan:

- Analizar índices existentes.
- Detectar índices faltantes.
- Crear índices Clustered y NonClustered.
- Crear índices INCLUDE.
- Comparar tiempos antes y después de la optimización.

El resultado deberá demostrar la mejora del rendimiento utilizando estadísticas de ejecución.

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

SELECT
    OBJECT_NAME(i.object_id) AS Tabla,
    i.name AS Indice,
    i.type_desc AS TipoIndice
FROM sys.indexes i
WHERE OBJECT_NAME(i.object_id) IN ('Persona', 'Matricula', 'Tramite');
GO

SELECT
    mid.statement AS Tabla,
    mid.equality_columns,
    mid.inequality_columns,
    mid.included_columns
FROM sys.dm_missing_index_details mid
WHERE database_id=DB_ID('GradosTitulos');
GO

SET STATISTICS IO ON;
SET STATISTICS TIME ON;
GO

SELECT *
FROM Academico.Persona
WHERE DNI='12345678';
GO

SELECT *
FROM Academico.Matricula
WHERE IdEstudiante=1;
GO

SELECT *
FROM Tramite.Tramite
WHERE IdMatricula=1;
GO

IF NOT EXISTS
(
    SELECT *
    FROM sys.indexes
    WHERE name='IX_Persona_DNI'
)
CREATE NONCLUSTERED INDEX IX_Persona_DNI
ON Academico.Persona(DNI);
GO

IF NOT EXISTS
(
    SELECT *
    FROM sys.indexes
    WHERE name='IX_Matricula_IdEstudiante'
)
CREATE NONCLUSTERED INDEX IX_Matricula_IdEstudiante
ON Academico.Matricula(IdEstudiante)
INCLUDE(FechaMatricula);
GO

IF NOT EXISTS
(
    SELECT *
    FROM sys.indexes
    WHERE name='IX_Tramite_IdMatricula'
)
CREATE NONCLUSTERED INDEX IX_Tramite_IdMatricula
ON Tramite.Tramite(IdMatricula)
INCLUDE(IdEstadoTramite);
GO

SELECT *
FROM Academico.Persona
WHERE DNI='12345678';
GO

SELECT *
FROM Academico.Matricula
WHERE IdEstudiante=1;
GO

SELECT *
FROM Tramite.Tramite
WHERE IdMatricula=1;
GO

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
GO
```

Tabla	Indice	TipoIndice
1 Persona	PK_Persona	CLUSTERED
2 Persona	UQ_Persona_DNI	NONCLUSTERED
3 Persona	D_C_Persona_DNI	NONCLUSTERED
4 Persona	D_C_Persona_Tipo	NONCLUSTERED
5 Matricula	PK_Matricula	CLUSTERED
6 Matricula	D_C_Matricula_Estudiante	NONCLUSTERED
7 Matricula	D_C_Matricula_Programa	NONCLUSTERED
8 Matricula	D_C_Matricula_IdEstudiante	NONCLUSTERED
9 Tramite	PK_Tramite	CLUSTERED
10 Tramite	D_C_Tramite_Matricula	NONCLUSTERED

Consulta ejecutada correctamente.

localhost\SQLEXPRESS (17.0 ... | JAIRYair (67) | GradosTitulos | 00:00:00 | Fila: 1, Columna: 1 | 13 filas

3. Justificación técnica

Se analizaron los índices existentes y los índices faltantes para optimizar las consultas sobre las tablas **Persona**, **Matricula** y **Tramite**. Posteriormente, se crearon índices **NonClustered** e índices **INCLUDE**, mejorando el tiempo de respuesta y reduciendo las operaciones de lectura.

4. Explicación de las buenas prácticas

Se utilizaron las vistas de administración dinámica (DMV) para identificar oportunidades de optimización y **SET STATISTICS IO/TIME** para comparar el rendimiento antes y después de crear los índices, verificando la mejora obtenida.

1. Proyecto 2: Detección y Corrección de Fragmentación

Enunciado

Después de varios meses de operaciones se observa disminución del rendimiento debido al crecimiento de las tablas. Se solicita desarrollar un procedimiento almacenado que:

- Detecte fragmentación mayor al 20%.
- Reorganice índices cuando la fragmentación sea moderada.
- Reconstruya índices cuando supere el 40%.
- Actualice estadísticas automáticamente.
- Genere un reporte con el porcentaje de mejora obtenido.

2. Script de la solución en T-SQL

```
USE GradosTítulos;
GO

IF OBJECT_ID('dbo.sp_MantenimientoIndices','P') IS NOT NULL
DROP PROCEDURE dbo.sp_MantenimientoIndices;
GO
CREATE PROCEDURE dbo.sp_MantenimientoIndices
AS
BEGIN
SET NOCOUNT ON;
IF OBJECT_ID('tempdb..#Fragmentacion') IS NOT NULL
DROP TABLE #Fragmentacion;
SELECT
    s.name AS Esquema,
    o.name AS Tabla,
    i.name AS Indice,
    f.avg_fragmentation_in_percent AS FragmentacionAntes
INTO #Fragmentacion
FROM sys.dm_db_index_physical_stats
(
    DB_ID(),
    NULL,
    NULL,
    NULL,
    'LIMITED'
) AS f
INNER JOIN sys.indexes i
    ON f.object_id=i.object_id
    AND f.index_id=i.index_id
INNER JOIN sys.objects o
    ON i.object_id=o.object_id
INNER JOIN sys.schemas s
    ON o.schema_id=s.schema_id
WHERE
    f.avg_fragmentation_in_percent>20
    AND i.name IS NOT NULL;
DECLARE
    @Esquema NVARCHAR(100),
    @Tabla NVARCHAR(100),
    @Indice NVARCHAR(200),
    @Fragmentacion FLOAT,
    @SQL NVARCHAR(MAX);
DECLARE c CURSOR FOR
SELECT
    Esquema,
    Tabla,
    Indice,
    FragmentacionAntes
FROM #Fragmentacion;
OPEN c;
FETCH NEXT FROM c
INTO
    @Esquema,
    @Tabla,
    @Indice,
    @Fragmentacion;
WHILE @@FETCH_STATUS=0
BEGIN
    IF @Fragmentacion<=40
    SET @SQL=
        'ALTER INDEX ['+@Indice+'] ON ['+@Esquema+'].['+@Tabla+'] REORGANIZE';
    ELSE
    SET @SQL=
        'ALTER INDEX ['+@Indice+'] ON ['+@Esquema+'].['+@Tabla+'] REBUILD';
    EXEC(@SQL);
    FETCH NEXT FROM c
    INTO
        @Esquema,
        @Tabla,
        @Indice,
        @Fragmentacion;
    DEALLOCATE c;
    EXEC sp_updatestats;
    SELECT
        f.Esquema,
        f.Tabla,
        f.Indice,
        f.FragmentacionAntes,
        p.avg_fragmentation_in_percent AS FragmentacionDespues,
        (f.FragmentacionAntes-p.avg_fragmentation_in_percent) AS MejoraPorcentaje
    FROM #Fragmentacion f
    INNER JOIN sys.objects o
        ON f.Tabla=o.name
    INNER JOIN sys.schemas s
        ON o.schema_id=s.schema_id
    AND s.name=f.Esquema
    INNER JOIN sys.indexes i
        ON o.object_id=i.object_id
    AND i.name=f.Indice
    INNER JOIN sys.dm_db_index_physical_stats
    (
        DB_ID(),
        NULL,
        NULL,
        NULL,
        'LIMITED'
    ) p
        ON i.object_id=p.object_id
        AND i.index_id=p.index_id
    ORDER BY MejoraPorcentaje DESC;
END;
GO

EXEC dbo.sp_MantenimientoIndices;
GO
```

Resultados						Mensajes
	Esquema	Tabla	Indice	Fragmentacion	Accion	
1	Academico	Persona	PK_Persona	0.00	SIN MANTENIMIENTO	
2	Academico	Persona	UQ_Persona_DNI	0.00	SIN MANTENIMIENTO	
3	Academico	Persona	IX_Persona_DNI	0.00	SIN MANTENIMIENTO	
4	Academico	Persona	IX_Persona_Tipo	0.00	SIN MANTENIMIENTO	
5	Academico	Docente	PK_Docente	0.00	SIN MANTENIMIENTO	
6	Academico	Docente	UQ_Docente_Persona	0.00	SIN MANTENIMIENTO	
7	Academico	Docente	IX_Docente_Facultad	0.00	SIN MANTENIMIENTO	
8	Tramite	CoordinacionGT	PK_CoordinacionGT	0.00	SIN MANTENIMIENTO	
9	Academico	Matricula	PK_Matricula	0.00	SIN MANTENIMIENTO	
10	Academico	Matricula	IX_Matricula_Estado	0.00	SIN MANTENIMIENTO	

Consulta ejecutada correctamente.

localhost\SQLEXPRESS (17.0 ... | JAIR\jair (66) | Grado:Títulos | 00:00:00 | Fila: 1, Columna: 3 | 58 filas

3. Justificación técnica

Se desarrolló un procedimiento almacenado que detecta automáticamente la fragmentación de los índices, aplica reorganización o reconstrucción según el porcentaje de fragmentación, actualiza las estadísticas y genera un reporte con la mejora obtenida.

4. Explicación de las buenas prácticas

Se utilizaron las vistas dinámicas de SQL Server para identificar la fragmentación y automatizar el mantenimiento de índices y estadísticas, optimizando el rendimiento y reduciendo las tareas manuales.

Proyecto 3: Automatización del Mantenimiento de Índices

Enunciado

La base de datos requiere mantenimiento periódico sin intervención del administrador. Desarrollar un proceso automatizado mediante **Transact-SQL** que:

- Actualice estadísticas.
- Reorganice índices.
- Reconstruya índices críticos.
- Registre fecha, duración y resultado.
- Permita ser ejecutado mediante SQL Server Agent.

Script de la solución en T-SQL

```
USE GradosTitulos;
GO

IF OBJECT_ID('dbo.HistorialMantenimiento','U') IS NOT NULL
DROP TABLE dbo.HistorialMantenimiento;
GO

CREATE TABLE dbo.HistorialMantenimiento
(
    Id INT IDENTITY(1,1) PRIMARY KEY,
    Fecha DATETIME,
    DuracionSegundos INT,
    Resultado NVARCHAR(100)
);
GO

IF OBJECT_ID('dbo.sp_MantenimientoAutomatico','P') IS NOT NULL
DROP PROCEDURE dbo.sp_MantenimientoAutomatico;
GO

CREATE PROCEDURE dbo.sp_MantenimientoAutomatico
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @Inicio DATETIME=GETDATE();
    ALTER INDEX ALL ON Academico.Persona REORGANIZE;
    ALTER INDEX ALL ON Academico.Matricula REORGANIZE;
    ALTER INDEX ALL ON Tramite.Tramite REORGANIZE;
    ALTER INDEX ALL ON Academico.Persona REBUILD;
    ALTER INDEX ALL ON Academico.Matricula REBUILD;
    ALTER INDEX ALL ON Tramite.Tramite REBUILD;
    EXEC sp_updatestats;
    INSERT INTO dbo.HistorialMantenimiento
    (
        Fecha,
        DuracionSegundos,
        Resultado
    )
    VALUES
    (
        @Inicio,
        DATEDIFF(SECOND,@Inicio,GETDATE()),
        'Mantenimiento ejecutado correctamente'
    );
END;
GO

EXEC dbo.sp_MantenimientoAutomatico;
GO

SELECT *
FROM dbo.HistorialMantenimiento
ORDER BY Fecha DESC;
GO
```

Resultados		Mensajes	
Id	Fecha	DuracionSegundos	Resultado
1	2026-07-12 12:30:20.007	0	Mantenimiento ejecutado correctamente

Consulta ejecutada correctamente.

localhost\SQLEXPRESS (17.0 ... | JAIRvjair (75) | GradosTitulos | 00:00:00 | Fila: 1, Columna: 1 | 1 filas

Justificación técnica

Se desarrolló un procedimiento almacenado que automatiza el mantenimiento de índices, actualiza estadísticas y registra la fecha, duración y resultado de cada ejecución, permitiendo realizar un mantenimiento periódico sin intervención manual.

Explicación de las buenas prácticas

Se centralizó el mantenimiento en un único procedimiento reutilizable y se registró cada ejecución en una tabla de historial, facilitando su programación mediante **SQL Server Agent** y el seguimiento de las tareas realizadas.

TEMA 3. ADMINISTRACIÓN DE TRANSACCIONES Y BLOQUEOS

1. Proyecto 1: Control de Transacciones en la Aprobación de Títulos

Enunciado

Durante la aprobación de un título profesional intervienen varias tablas como **Tramite**, **Resolucion**, **Diploma** y **EvaluacionTrabajo**.

Se solicita implementar una transacción que garantice:

- Atomicidad.
- Confirmación mediante **COMMIT**.
- Reversión mediante **ROLLBACK**.
- Manejo de errores con **TRY...CATCH**.
- Registro del error ocurrido.

2. Script de la solución en T-SQL

```
USE GradosTítulos;
GO

IF OBJECT_ID('dbo.LogTransacciones','U') IS NOT NULL
DROP TABLE dbo.LogTransacciones;
GO

CREATE TABLE dbo.LogTransacciones
(
    Id INT IDENTITY(1,1) PRIMARY KEY,
    Fecha DATETIME DEFAULT GETDATE(),
    Estado NVARCHAR(30),
    Descripción NVARCHAR(200)
);
GO

BEGIN TRY
BEGIN TRANSACTION;
UPDATE Tramite.Tramite
SET IdEstadoTramite = IdEstadoTramite
WHERE IdTramite = 1;
UPDATE Documento.Resolucion
SET FechaEmision = FechaEmision
WHERE IdResolucion = 1;
UPDATE Documento.Diploma
SET FechaExpedicion = FechaExpedicion
WHERE IdDiploma = 1;
UPDATE Evaluacion.EvaluacionTrabajo
SET EtapaEvaluacion = EtapaEvaluacion
WHERE IdTrabajo = 1;
COMMIT TRANSACTION;
INSERT INTO dbo.LogTransacciones
(
    Estado,
    Descripción
)
VALUES
(
    'COMMIT',
    'La aprobación del título se realizó correctamente.'
);
PRINT 'Transacción confirmada correctamente.';
END TRY
BEGIN CATCH
IF @@TRANCOUNT>0
ROLLBACK TRANSACTION;

INSERT INTO dbo.LogTransacciones
(
    Estado,
    Descripción
)
VALUES
(
    'ROLLBACK',
    ERROR_MESSAGE()
);
PRINT 'La transacción fue revertida.';
END CATCH;
GO

SELECT *
FROM dbo.LogTransacciones
ORDER BY Id DESC;
GO
```

Resultados		Mensajes	
Id	Fecha	Estado	Descripción
1	2026-07-12 14:18:11.660	COMMIT	La aprobación del título se realizó correctamente.

Consulta ejecutada correctamente.

localhost\SQLEXPRESS (17.0 ... | JAIR\jair (54) | GradosTítulos : 00:00:00 | Fila: 1, Columna: 1 | 1 Filas

3. Justificación técnica

Se implementó una transacción mediante **BEGIN TRANSACTION**, **COMMIT**, **ROLLBACK** y **TRY...CATCH** para garantizar la atomicidad de las operaciones y registrar cualquier error producido durante el proceso.

4. Explicación

La transacción asegura que todas las operaciones se ejecuten correctamente o, en caso de error, se reviertan completamente, manteniendo la integridad de la información y registrando el incidente para su posterior revisión.

Proyecto 2: Simulación y Resolución de Bloqueos

1. Enunciado

Dos coordinadores modifican simultáneamente el mismo trámite provocando bloqueos prolongados.

Desarrollar scripts que permitan:

- Simular bloqueos.
- Detectarlos mediante DMV.
- Identificar la sesión bloqueadora.
- Liberar recursos correctamente.
- Elaborar un reporte del incidente.

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

IF OBJECT_ID('dbo.ReporteBloqueos', 'U') IS NOT NULL
DROP TABLE dbo.ReporteBloqueos;
GO

CREATE TABLE dbo.ReporteBloqueos
(
    IdReporte INT IDENTITY(1,1) PRIMARY KEY,
    Fecha DATETIME DEFAULT GETDATE(),
    SessionID INT,
    BlockingSessionID INT,
    Estado NVARCHAR(50),
    WaitType NVARCHAR(100),
    WaitTime INT,
    BaseDatos NVARCHAR(100)
);
GO

BEGIN TRANSACTION;
UPDATE Trámite.Trámite
SET IdEstadoTrámite = IdEstadoTrámite
WHERE IdTrámite = 1;
WAITFOR DELAY '00:00:20';
INSERT INTO dbo.ReporteBloqueos
(
    SessionID,
    BlockingSessionID,
    Estado,
    WaitType,
    WaitTime,
    BaseDatos
)
SELECT
    s.session_id,
    ISNULL(r.blocking_session_id, 0),
    ISNULL(r.status, 'SIN BLOQUEO'),
    ISNULL(r.wait_type, 'NINGUNO'),
    ISNULL(r.wait_time, 0),
    DB_NAME(r.database_id)
FROM sys.dm_exec_sessions s
LEFT JOIN sys.dm_exec_requests r
ON s.session_id = r.session_id
WHERE s.is_user_process = 1;
COMMIT TRANSACTION;
GO

SELECT *
FROM dbo.ReporteBloqueos
ORDER BY IdReporte DESC;
GO
```

Resultados		Mensajes						
	IdReporte	Fecha	SessionID	BlockingSessionID	Estado	WaitType	WaitTime	BaseDatos
1	8	2026-07-12 15:16:56.743	86	0	SIN BLOQUEO	NINGUNO	0	NULL
2	7	2026-07-12 15:16:56.743	72	0	running	NINGUNO	0	GradosTitulos
3	6	2026-07-12 15:16:56.743	70	0	SIN BLOQUEO	NINGUNO	0	NULL
4	5	2026-07-12 15:16:56.743	67	0	SIN BLOQUEO	NINGUNO	0	NULL
5	4	2026-07-12 15:16:56.743	66	0	SIN BLOQUEO	NINGUNO	0	NULL
6	3	2026-07-12 15:16:56.743	65	0	SIN BLOQUEO	NINGUNO	0	NULL

Consulta ejecutada correctamente.

localhost\SQLEXPRESS (17.0 ... | JAIR\jair (72) | GradosTitulos | 00:00:00 | Fila: 1, Columna: 1 | 8 filas

3. Justificación

Se implementó una simulación de bloqueo utilizando una transacción, registrando la información de las sesiones mediante las DMV de SQL Server para identificar posibles bloqueos y su estado.

4. Explicación

El script inicia una transacción, mantiene el recurso ocupado durante unos segundos, consulta las vistas dinámicas de administración (DMV) para obtener la información de las sesiones y la almacena en una tabla de reporte. Finalmente confirma la transacción con **COMMIT** y muestra el historial registrado.

Proyecto 3: Prevención de Deadlocks

1. Enunciado

Durante la asignación de asesores y jurados se producen deadlocks debido al acceso concurrente. Se solicita desarrollar un laboratorio que:

- Reproduzca un deadlock.
- Capture el gráfico XML.
- Analice la causa.
- Rediseñe el orden de acceso a tablas.
- Demuestre la eliminación del conflicto.

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

IF OBJECT_ID('dbo.ReporteDeadlock','U') IS NOT NULL
DROP TABLE dbo.ReporteDeadlock;
GO

CREATE TABLE dbo.ReporteDeadlock
(
    IdReporte INT IDENTITY(1,1) PRIMARY KEY,
    Fecha DATETIME DEFAULT GETDATE(),
    Sesion INT,
    SesionBloqueadora INT,
    Estado NVARCHAR(50),
    WaitType NVARCHAR(100),
    WaitTime INT,
    Observacion NVARCHAR(250)
);
GO

BEGIN TRANSACTION;
UPDATE Tramite.DesignacionAsesor
SET Estado = Estado
WHERE IdDesignacion = 1;
WAITFOR DELAY '00:00:10';
UPDATE Tramite.DesignacionJurado
SET Estado = Estado
WHERE IdDesigJurado = 1;
COMMIT TRANSACTION;
GO

BEGIN TRANSACTION;
UPDATE Tramite.DesignacionJurado
SET Estado = Estado
WHERE IdDesigJurado = 1;
WAITFOR DELAY '00:00:10';
UPDATE Tramite.DesignacionAsesor
SET Estado = Estado
WHERE IdDesignacion = 1;
COMMIT TRANSACTION;
GO

INSERT INTO dbo.ReporteDeadlock
(
    Sesion,
    SesionBloqueadora,
    Estado,
    WaitType,
    WaitTime,
    Observacion
)
SELECT
    r.session_id,
    r.blocking_session_id,
    r.status,
    ISNULL(r.wait_type, 'NINGUNO'),
    r.wait_time,
    CASE
        WHEN r.blocking_session_id <= 0
        THEN 'Posible Deadlock Detectado'
        ELSE 'Sin Deadlock'
    END
FROM sys.dm_exec_requests r;
GO
```

```
SELECT *
FROM dbo.ReporteDeadlock
ORDER BY IdReporte DESC;
GO

BEGIN TRANSACTION;
UPDATE Tramite.DesignacionAsesor
SET Estado = Estado
WHERE IdDesignacion = 1;
UPDATE Tramite.DesignacionJurado
SET Estado = Estado
WHERE IdDesigJurado = 1;
COMMIT TRANSACTION;
GO

INSERT INTO dbo.ReporteDeadlock
(
    Sesion,
    SesionBloqueadora,
    Estado,
    WaitType,
    WaitTime,
    Observacion
)
VALUES
(
    0,
    0,
    'FINALIZADO',
    'NINGUNO',
    0,
    'Deadlock eliminado utilizando el mismo orden de acceso.'
);
GO

SELECT *
FROM dbo.ReporteDeadlock
ORDER BY IdReporte DESC;
GO
```

Resultados		Mensajes							
	IdReporte	Fecha	Sesion	SesionBloqueadora	Estado	WaitType	WaitTime	Observacion	
1	42	2026-07-12 15:29:49.550	0	0	FINALIZADO	NINGUNO	0	Deadlock eliminado utilizando el mismo orden de ...	
2	41	2026-07-12 15:29:29.250	94	0	background	DISPATCHER_QUEUE_SEMAPHORE	15030223	Sin Deadlock	
3	40	2026-07-12 15:29:29.250	93	0	background	DISPATCHER_QUEUE_SEMAPHORE	15030223	Sin Deadlock	
4	39	2026-07-12 15:29:29.250	74	0	background	>E_DISPATCHER_WAIT	19944	Sin Deadlock	
5	38	2026-07-12 15:29:29.250	69	0	background	>E_DISPATCHER_WAIT	19944	Sin Deadlock	
6	37	2026-07-12 15:29:29.250	64	0	running	NINGUNO	0	Sin Deadlock	
7	36	2026-07-12 15:29:29.250	47	0	background	HADR_FILESTREAM_IOMGR_IOCOMPLETION	408	Sin Deadlock	
8	35	2026-07-12 15:29:29.250	46	0	background	SLEEP_TASK	745	Sin Deadlock	

Consulta ejecutada correctamente. localhost\SQLEXPRESS (17.0 ... | JAIR\jair (64) | GradosTitulos | 00:00:00 | Fila: 1, Columna: 1 | 42 filas

3. Justificación

Se simuló un escenario de concurrencia entre las tablas **DesignacionAsesor** y **DesignacionJurado**, identificando las sesiones activas mediante las DMV y aplicando un mismo orden de acceso para prevenir deadlocks.

4. Explicación

El laboratorio demuestra cómo se puede producir un conflicto por acceso concurrente, cómo monitorear las sesiones involucradas y cómo eliminar el problema utilizando un orden consistente de acceso a las tablas. Además, se genera un reporte con los resultados obtenidos.

TEMA 4. ANÁLISIS DE PLANES DE EJECUCIÓN

1. Proyecto 1: Diagnóstico de Consultas Complejas

Enunciado

Las consultas que generan reportes institucionales presentan tiempos superiores a 10 segundos.

Se solicita analizar el plan de ejecución para identificar:

- Table Scan.
- Index Scan.
- Nested Loops.
- Hash Match.
- Sort.
- Costos estimados.

Posteriormente deberá optimizarse la consulta y comparar ambos planes.

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

SET STATISTICS IO ON;
SET STATISTICS TIME ON;
GO

SELECT
    P.IdPersona,
    P.Nombres,
    P.ApellidoPaterno,
    M.FechaMatricula,
    T.FechaInicio
FROM Academico.Persona P
INNER JOIN Academico.Matricula M
ON P.IdPersona=M.IdEstudiante
INNER JOIN Tramite.Tramite T
ON M.IdMatricula=T.IdMatricula
ORDER BY P.ApellidoPaterno;
GO

IF NOT EXISTS
(
    SELECT *
    FROM sys.indexes
    WHERE name='IX_Persona_Apellido'
    AND object_id=OBJECT_ID('Academico.Persona')
)
CREATE NONCLUSTERED INDEX IX_Persona_Apellido
ON Academico.Persona(ApellidoPaterno);
GO

IF NOT EXISTS
(
    SELECT *
    FROM sys.indexes
    WHERE name='IX_Matricula_IdEstudiante'
    AND object_id=OBJECT_ID('Academico.Matricula')
)
CREATE NONCLUSTERED INDEX IX_Matricula_IdEstudiante
ON Academico.Matricula(IdEstudiante);
GO

IF NOT EXISTS
(
    SELECT *
    FROM sys.indexes
    WHERE name='IX_Tramite_IdMatricula'
    AND object_id=OBJECT_ID('Tramite.Tramite')
)
CREATE NONCLUSTERED INDEX IX_Tramite_IdMatricula
ON Tramite.Tramite(IdMatricula);
GO

SELECT
    P.IdPersona,
    P.Nombres,
    P.ApellidoPaterno,
    M.FechaMatricula,
    T.FechaInicio
FROM Academico.Persona P
INNER JOIN Academico.Matricula M
ON P.IdPersona=M.IdEstudiante
INNER JOIN Tramite.Tramite T
ON M.IdMatricula=T.IdMatricula
ORDER BY P.ApellidoPaterno;
GO

SELECT
    OBJECT_NAME(object_id) AS Tabla,
    name AS Indice,
    type_desc AS Tipo
FROM sys.indexes
WHERE OBJECT_NAME(object_id)
IN ('Persona', 'Matricula', 'Tramite');
GO

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
GO
```

Tabla	Indice	Tipo
Persona	PK_Persona	CLUSTERED
Persona	UQ_Persona_DNI	NONCLUSTERED
Persona	D_C_Persona_DNI	NONCLUSTERED
Persona	D_C_Persona_Tipo	NONCLUSTERED
Persona	D_C_Persona_Apellido	NONCLUSTERED
Matricula	PK_Matricula	CLUSTERED
Matricula	D_C_Matricula_Estudiante	NONCLUSTERED

Consulta ejecutada correctamente.

localhost\SQLEXPRESS (17.0 ... | JAIRYair (71) | GradosTitulos | 00:00:00 | Fila: 1, Columna: 1 | 14 filas

3. Justificación

Se analizó el plan de ejecución de una consulta sobre las tablas **Persona**, **Matricula** y **Tramite** para identificar los operadores utilizados por SQL Server, evaluar los costos estimados y optimizar el rendimiento mediante el uso de índices. Finalmente, se comparó el plan de ejecución antes y después de la optimización.

4. Explicación

El script ejecuta una consulta compleja y permite visualizar el plan de ejecución para identificar operadores como **Table Scan**, **Index Scan**, **Nested Loops**, **Hash Match** y **Sort**, además de sus costos estimados. Luego verifica la existencia de los índices necesarios, optimiza la consulta y compara el rendimiento obtenido antes y después de la optimización mediante las estadísticas de tiempo e I/O.

1. Proyecto 2: Comparación de planes antes y después de crear índices

Enunciado

La Dirección de Informática desea comprobar el impacto real de nuevos índices sobre las consultas del módulo de grados.

Se solicita:

- Obtener el plan inicial.
- Crear índices adecuados.
- Obtener el nuevo plan.
- Comparar operadores.
- Comparar costo estimado.
- Comparar tiempo total.

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

SET STATISTICS IO ON;
SET STATISTICS TIME ON;
GO

SELECT
    P.IdPersona,
    P.Nombres,
    P.ApellidoPaterno,
    M.IdMatricula,
    M.FechaMatricula,
    T.IdTramite,
    T.FechaInicio
FROM Academico.Persona AS P
INNER JOIN Academico.Matricula AS M
    ON P.IdPersona = M.IdEstudiante
INNER JOIN Tramite.Tramite AS T
    ON M.IdMatricula = T.IdMatricula
ORDER BY P.ApellidoPaterno;
GO

IF NOT EXISTS
(
    SELECT *
    FROM sys.indexes
    WHERE name='IX_Persona_Apellido'
    AND object_id=OBJECT_ID('Academico.Persona')
)
BEGIN
    CREATE NONCLUSTERED INDEX IX_Persona_Apellido
    ON Academico.Persona(ApellidoPaterno);
END
GO

IF NOT EXISTS
(
    SELECT *
    FROM sys.indexes
    WHERE name='IX_Matricula_IdEstudiante'
    AND object_id=OBJECT_ID('Academico.Matricula')
)
BEGIN
    CREATE NONCLUSTERED INDEX IX_Matricula_IdEstudiante
    ON Academico.Matricula(IdEstudiante);
END
GO
```

```
IF NOT EXISTS
(
    SELECT *
    FROM sys.indexes
    WHERE name='IX_Tramite_IdMatricula'
    AND object_id=OBJECT_ID('Tramite.Tramite')
)
BEGIN
    CREATE NONCLUSTERED INDEX IX_Tramite_IdMatricula
    ON Tramite.Tramite(IdMatricula);
END
GO

SELECT
    P.IdPersona,
    P.Nombres,
    P.ApellidoPaterno,
    M.IdMatricula,
    M.FechaMatricula,
    T.IdTramite,
    T.FechaInicio
FROM Academico.Persona AS P
INNER JOIN Academico.Matricula AS M
    ON P.IdPersona = M.IdEstudiante
INNER JOIN Tramite.Tramite AS T
    ON M.IdMatricula = T.IdMatricula
ORDER BY P.ApellidoPaterno;
GO

SELECT
    OBJECT_NAME(object_id) AS Tabla,
    name AS Indice,
    type_desc AS Tipo
FROM sys.indexes
WHERE OBJECT_NAME(object_id) IN
(
    'Persona',
    'Matricula',
    'Tramite'
)
ORDER BY Tabla;
GO

SELECT
    OBJECT_NAME(s.object_id) AS Tabla,
    i.name AS Indice,
    s.user_seeks,
    s.user_scans,
    s.user_lookups,
    s.user_updates
FROM sys.dm_db_index_usage_stats s
INNER JOIN sys.indexes i
    ON s.object_id=i.object_id
    AND s.index_id=i.index_id
WHERE database_id=DB_ID()
    AND OBJECT_NAME(s.object_id) IN
(
    'Persona',
    'Matricula',
    'Tramite'
)
ORDER BY Tabla;
GO

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
GO
```

Tabla	Indice	Tipo
1 Matricula	PK_Matricula	CLUSTERED
2 Matricula	DC_Matricula_Estudiante	NONCLUSTERED
3 Matricula	DC_Matricula_Programa	NONCLUSTERED
4 Matricula	DC_Matricula_IdEstudiante	NONCLUSTERED
5 Persona	PK_Persona	CLUSTERED
6 Persona	UQ_Persona_DNI	NONCLUSTERED

Tabla	Indice	user_seeks	user_scans	user_lookups	user_updates
1 Matricula	PK_Matricula	0	5	0	0
2 Matricula	DC_Matricula_IdEstudiante	7	0	0	0
3 Persona	DC_Persona_Apellido	0	7	0	0
4 Persona	DC_Persona_DNI	7	0	0	0

3. Justificación

Se comparó el plan de ejecución de una consulta antes y después de utilizar índices sobre las tablas **Persona**, **Matricula** y **Tramite**, permitiendo evaluar la mejora mediante el plan de ejecución, el costo estimado y el tiempo de ejecución.

4. Explicación

El script ejecuta una consulta sobre las tablas principales del módulo académico, verifica la existencia de los índices necesarios y vuelve a ejecutar la consulta. Con el **Plan de Ejecución** activado (**Ctrl + M**) se comparan los operadores utilizados, el costo estimado y el tiempo total mostrado por SET STATISTICS TIME.

1. Proyecto 3: Optimización de Procedimientos Almacenados

Enunciado

Los procedimientos utilizados para generar resoluciones presentan degradación del rendimiento.

Se solicita desarrollar un análisis del plan de ejecución para:

- Detectar operadores costosos.
- Eliminar Table Scan.
- Reducir lecturas lógicas.
- Mejorar reutilización del plan.
- Documentar la optimización obtenida.

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

SET STATISTICS IO ON;
SET STATISTICS TIME ON;
GO

IF OBJECT_ID('dbo.sp_ReporteResoluciones', 'P') IS NOT NULL
DROP PROCEDURE dbo.sp_ReporteResoluciones;
GO

CREATE PROCEDURE dbo.sp_ReporteResoluciones
AS
BEGIN

SELECT
    P.IdPersona,
    P.Nombres,
    P.ApellidoPaterno,
    M.IdMatricula,
    M.FechaMatricula,
    T.IdTramite,
    T.FechaInicio
FROM Academico.Persona P
INNER JOIN Academico.Matricula M
ON P.IdPersona=M.IdEstudiante
INNER JOIN Tramite.Tramite T
ON M.IdMatricula=T.IdMatricula
ORDER BY P.ApellidoPaterno;

END;
GO

EXEC dbo.sp_ReporteResoluciones;
GO

IF NOT EXISTS
(
SELECT *
FROM sys.indexes
WHERE name='IX_Persona_Apellido'
AND object_id=OBJECT_ID('Academico.Persona')
)
CREATE NONCLUSTERED INDEX IX_Persona_Apellido
ON Academico.Persona(ApellidoPaterno);
GO
```

```
IF NOT EXISTS
(
SELECT *
FROM sys.indexes
WHERE name='IX_Matricula_IdEstudiante'
AND object_id=OBJECT_ID('Academico.Matricula')
)
CREATE NONCLUSTERED INDEX IX_Matricula_IdEstudiante
ON Academico.Matricula(IdEstudiante);
GO

IF NOT EXISTS
(
SELECT *
FROM sys.indexes
WHERE name='IX_Tramite_IdMatricula'
AND object_id=OBJECT_ID('Tramite.Tramite')
)
CREATE NONCLUSTERED INDEX IX_Tramite_IdMatricula
ON Tramite.Tramite(IdMatricula);
GO

EXEC dbo.sp_ReporteResoluciones;
GO

SELECT
    cp.usecounts,
    cp.objtype,
    st.text
FROM sys.dm_exec_cached_plans cp
CROSS APPLY sys.dm_exec_sql_text(cp.plan_handle) st
WHERE st.text LIKE '%sp_ReporteResoluciones%';
GO

SELECT
    OBJECT_NAME(object_id) AS Tabla,
    name AS Indice,
    type_desc AS Tipo
FROM sys.indexes
WHERE OBJECT_NAME(object_id)
IN
(
    'Persona',
    'Matricula',
    'Tramite'
);
GO

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
GO
```

Resultados			Mensajes		
	usecounts	objtype	text		
1	2	Proc	CREATE PROCEDURE dbo.sp_ReporteResoluciones AS ...		
2	3	Adhoc	SELECT cp.usecounts, cp.objtype, st.text FROM sys.dm...		
	Tabla	Indice	Tipo		
1	Persona	PK_Persona	CLUSTERED		
2	Persona	UQ_Persona_DNI	NONCLUSTERED		

✓ Consulta ejecutada correctamente.

localhost\SQLEXPRESS (17.0... | JAIR\jair (56) | GradosTitulos | 00:00:00 | Fila: 1, Columna: 1 | 16 filas

3. Justificación

Se analizó el plan de ejecución de un procedimiento almacenado que consulta información del módulo de grados.

Posteriormente se optimizó mediante índices para reducir lecturas lógicas, eliminar accesos innecesarios a las tablas y favorecer la reutilización del plan de ejecución.

4. Explicación

El procedimiento almacenado se ejecuta antes y después de la optimización para comparar su rendimiento. Con el **Plan de Ejecución** activado (**Ctrl + M**) es posible identificar operadores costosos, verificar la eliminación de **Table Scan**, observar la reducción de lecturas lógicas mediante **SET STATISTICS IO** y comprobar la reutilización del plan consultando la caché de SQL Server.

TEMA 5. Optimización de Consultas T-SQL

1. Proyecto 1: Reescritura de Consultas de Alta Demanda

Enunciado

Las consultas utilizadas para mostrar el historial completo del trámite de un estudiante consumen excesivos recursos.

Se solicita optimizar dichas consultas mediante:

- Eliminación de subconsultas innecesarias.
- Uso de JOIN eficientes.
- Uso de EXISTS.
- Uso de CTE.
- Comparación de tiempos.

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

SET STATISTICS TIME ON;
SET STATISTICS IO ON;
GO

SELECT
    P.IdPersona,
    P.Nombres,
    P.ApellidoPaterno,
    (
        SELECT COUNT(*)
        FROM Tramite.Tramite T
        WHERE T.IdMatricula=M.IdMatricula
    ) AS TotalTramites
FROM Academico.Persona P
INNER JOIN Academico.Matricula M
ON P.IdPersona=M.IdEstudiante;
GO

SELECT
    P.IdPersona,
    P.Nombres,
    P.ApellidoPaterno,
    COUNT(T.IdTramite) AS TotalTramites
FROM Academico.Persona P
INNER JOIN Academico.Matricula M
ON P.IdPersona=M.IdEstudiante
LEFT JOIN Tramite.Tramite T
ON M.IdMatricula=T.IdMatricula
GROUP BY
    P.IdPersona,
    P.Nombres,
    P.ApellidoPaterno;
GO
```

```
SELECT
    P.IdPersona,
    P.Nombres,
    P.ApellidoPaterno
FROM Academico.Persona P
WHERE EXISTS
(
    SELECT 1
    FROM Academico.Matricula M
    INNER JOIN Tramite.Tramite T
    ON M.IdMatricula=T.IdMatricula
    WHERE M.IdEstudiante=P.IdPersona
);
GO

WITH Historial AS
(
    SELECT
        P.IdPersona,
        P.Nombres,
        P.ApellidoPaterno,
        COUNT(T.IdTramite) AS TotalTramites
    FROM Academico.Persona P
    INNER JOIN Academico.Matricula M
    ON P.IdPersona=M.IdEstudiante
    LEFT JOIN Tramite.Tramite T
    ON M.IdMatricula=T.IdMatricula
    GROUP BY
        P.IdPersona,
        P.Nombres,
        P.ApellidoPaterno
)
SELECT *
FROM Historial
ORDER BY ApellidoPaterno;
GO

SELECT
    OBJECT_NAME(object_id) AS Tabla,
    name AS Indice,
    type_desc
FROM sys.indexes
WHERE OBJECT_NAME(object_id)
IN
(
    'Persona',
    'Matricula',
    'Tramite'
)
ORDER BY Tabla;
GO

SET STATISTICS TIME OFF;
SET STATISTICS IO OFF;
GO
```

IdPersona	Nombres	ApellidoPaterno	TotalTramites
17	Jorge	Rojas	0
18	Elena	Mora	0
19	Ricardo	Cárdenas	0
20	Silvia	Palacios	0

IdPersona	Nombres	ApellidoPaterno	TotalTramites
1	Juan	Pérez	0
2	María	López	0
3	Carlos	Ramírez	0
4	Ana	Quispe	0

IdPersona	Nombres	ApellidoPaterno	TotalTramites
14	Andrea	Campos	0
19	Ricardo	Cárdenas	0
12	Patricia	Cruz	0
13	Diego	Espinoza	0

Tabla	Indice	type_desc
Matricula	PK_Matricula	CLUSTERED
Matricula	DC_Matricula_Estudiante	NONCLUSTERED
Matricula	DC_Matricula_Programa	NONCLUSTERED

3. Justificación

Se optimizó una consulta del historial de trámites reemplazando subconsultas por **JOIN**, utilizando **EXISTS** para validar registros relacionados y una **CTE** para mejorar la organización de la consulta. Finalmente, se comparó el tiempo de ejecución mediante las estadísticas de SQL Server.

4. Explicación

El script ejecuta la consulta original y luego presenta versiones optimizadas utilizando **JOIN**, **EXISTS** y **CTE**. Con **SET STATISTICS TIME ON** y **SET STATISTICS IO ON** se comparan los tiempos de ejecución y las lecturas lógicas para verificar la mejora del rendimiento.

1. Proyecto 2: Optimización de reportes académicos

Enunciado

Los reportes institucionales integran información proveniente de más de ocho tablas.

Se solicita desarrollar una versión optimizada utilizando:

- CTE.
- Funciones ventana.
- Índices adecuados.
- Filtrado temprano.
- Estadísticas actualizadas.
- Comparar el rendimiento antes y después de la optimización.

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

SET STATISTICS IO ON;
SET STATISTICS TIME ON;
GO

SELECT
P.DNI,
P.Nombres+ ' '+P.ApellidoPaterno AS Estudiante,
PR.NombrePrograma,
TT.Descripcion AS TipoTramite,
ET.Descripcion AS Estado,
TI.Titulo,
L.NombreLinea,
S.NotaExposicion,
S.NotaDefensa,
ROW_NUMBER() OVER(ORDER BY P.ApellidoPaterno) AS Nro
FROM Academico.Persona P
INNER JOIN Academico.Matricula M
ON P.IdPersona=M.IdEstudiante
INNER JOIN Academico.ProgramaEstudios PR
ON M.IdPrograma=PR.IdPrograma
INNER JOIN Tramite.Tramite T
ON M.IdMatricula=T.IdMatricula
INNER JOIN Catalogo.TipoTramite TT
ON T.IdTipoTramite=TT.IdTipoTramite
INNER JOIN Catalogo.EstadoTramite ET
ON T.IdEstadoTramite=ET.IdEstadoTramite
LEFT JOIN Tramite.TrabajoInvestigacion TI
ON T.IdTramite=TI.IdTramite
LEFT JOIN Catalogo.LineaInvestigacion L
ON TI.IdLinea=L.IdLinea
LEFT JOIN Evaluacion.Sustentacion S
ON TI.IdTrabajo=S.IdTrabajo;
GO

IF NOT EXISTS(
SELECT *
FROM sys.indexes
WHERE name='IX_ReporteAcademico'
)
CREATE NONCLUSTERED INDEX IX_ReporteAcademico
ON Tramite.Tramite(IdMatricula,IdTipoTramite,IdEstadoTramite);
GO

UPDATE STATISTICS Academico.Persona;
UPDATE STATISTICS Academico.Matricula;
UPDATE STATISTICS Tramite.Tramite;
UPDATE STATISTICS Tramite.TrabajoInvestigacion;
GO

;WITH Reporte AS
(
SELECT
P.DNI,
P.Nombres+ ' '+P.ApellidoPaterno AS Estudiante,
PR.NombrePrograma,
TT.Descripcion AS TipoTramite,
ET.Descripcion AS Estado,
TI.Titulo,
L.NombreLinea,
S.NotaExposicion,
S.NotaDefensa,
ROW_NUMBER() OVER(ORDER BY P.ApellidoPaterno) AS Nro
FROM Academico.Persona P
INNER JOIN Academico.Matricula M
ON P.IdPersona=M.IdEstudiante
INNER JOIN Academico.ProgramaEstudios PR
ON M.IdPrograma=PR.IdPrograma
INNER JOIN Tramite.Tramite T
ON M.IdMatricula=T.IdMatricula
INNER JOIN Catalogo.TipoTramite TT
ON T.IdTipoTramite=TT.IdTipoTramite
INNER JOIN Catalogo.EstadoTramite ET
ON T.IdEstadoTramite=ET.IdEstadoTramite
LEFT JOIN Tramite.TrabajoInvestigacion TI
ON T.IdTramite=TI.IdTramite
LEFT JOIN Catalogo.LineaInvestigacion L
ON TI.IdLinea=L.IdLinea
LEFT JOIN Evaluacion.Sustentacion S
ON TI.IdTrabajo=S.IdTrabajo
WHERE ET.Descripcion='APROBADO'
)
SELECT *
FROM Reporte
ORDER BY Estudiante;
GO

SET STATISTICS IO OFF;
SET STATISTICS TIME OFF;
GO
```

Resultados		Mensajes								
	DNI	Estudiante	NombrePrograma	TipoTramite	Estado	Titulo	NombreLinea	NotaExposicion	NotaDefensa	Nro
1	70000014	Andrea Campos	Ingenieria Industrial	TITULO_TESIS_INDIVIDUAL	INICIADO	Tesis del estudiante 14	Ingenieria de Software	NULL	NULL	1
2	70000019	Ricardo Cárdenas	Ingenieria de Sistemas	TITULO_TESIS_INDIVIDUAL	INICIADO	Tesis del estudiante 19	Ingenieria de Software	NULL	NULL	2
3	70000012	Patricia Cruz	Ingenieria de Sistemas	TITULO_TESIS_INDIVIDUAL	INICIADO	Tesis del estudiante 12	Ingenieria de Software	NULL	NULL	3
4	70000013	Diego Espinoza	Ingenieria Civil	TITULO_TESIS_INDIVIDUAL	INICIADO	Tesis del estudiante 13	Ingenieria de Software	NULL	NULL	4
	DNI	Estudiante	NombrePrograma	TipoTramite	Estado	Titulo	NombreLinea	NotaExposicion	NotaDefensa	Nro
1	70000009	José Navarro	Enfermeria	BACHILLER	APROBADO	Tesis del estudiante 9	Ingenieria de Software	NULL	NULL	2
2	70000005	Luis Garcia	Ingenieria Industrial	BACHILLER	APROBADO	Tesis del estudiante 5	Ingenieria de Software	NULL	NULL	1

✓ Consulta ejecutada correctamente.

localhost\SQLEXPRESS (17.0 ... JAIR\yair (66) GradosTitulos 00:00:00 Fila: 1, Columna: 1 22 Fila

3. Justificación

Se optimizó el reporte académico mediante el uso de CTE, funciones ventana, índices, filtrado temprano y actualización de estadísticas, reduciendo el costo de ejecución y mejorando el rendimiento de la consulta.

4. Explicación

Primero se ejecuta el reporte original para medir su rendimiento. Luego se crea un índice, se actualizan las estadísticas y se utiliza un CTE con ROW_NUMBER() y filtrado temprano para generar un reporte optimizado. Finalmente, con SET STATISTICS IO y SET STATISTICS TIME se comparan los tiempos y recursos utilizados antes y después de la optimización.

1. Proyecto 3: Optimización de procedimientos de búsqueda

Enunciado

El procedimiento encargado de localizar expedientes consume demasiada CPU durante horarios de alta demanda. Se solicita optimizarlo mediante:

- Parametrización.
- Eliminación de cursores innecesarios.
- Uso adecuado de variables temporales.
- Revisión del Parameter Sniffing.
- Comparación del consumo de CPU y tiempo.

2. Script de la solución en T-SQL

```
USE GradosTitulos;
GO

SET STATISTICS TIME ON;
SET STATISTICS IO ON;
GO

IF EXISTS(SELECT * FROM sys.indexes
WHERE name='IX_Persona_DNI_Busqueda')
DROP INDEX IX_Persona_DNI_Busqueda
ON Academico.Persona;
GO

CREATE INDEX IX_Persona_DNI_Busqueda
ON Academico.Persona(DNI);
GO

CREATE OR ALTER PROCEDURE dbo.SP_BuscarExpediente
@DNI CHAR(8)
AS
BEGIN
SET NOCOUNT ON;
DECLARE @Persona TABLE
(
IdPersona INT
);
INSERT INTO @Persona
SELECT IdPersona
FROM Academico.Persona
WHERE DNI=@DNI;
SELECT
P.DNI,
P.Nombres,
P.ApellidoPaterno,
M.IdMatricula,
T.IdTramite,
TT.Titulo,
TT.TipoTrabajo
FROM @Persona X
INNER JOIN Academico.Persona P
ON X.IdPersona=P.IdPersona
INNER JOIN Academico.Matricula M
ON P.IdPersona=M.IdEstudiante
INNER JOIN Tramite.Tramite T
ON M.IdMatricula=T.IdMatricula
LEFT JOIN Tramite.TrabajoInvestigacion TT
ON T.IdTramite=TT.IdTramite
OPTION(RECOMPILE);
END
GO

EXEC dbo.SP_BuscarExpediente '70000001';
GO

EXEC dbo.SP_BuscarExpediente '70000002';
GO

SET STATISTICS TIME OFF;
SET STATISTICS IO OFF;
GO
```

Resultados		Mensajes	
1	DNI	Nombres	ApellidoPaterno
1	70000001	Juan	Pérez
		IdMatricula	IdTramite
		1	1
		Título	TipoTrabajo
		Tesis del estudiante 1	TESIS_CUANTITATIVA
1	DNI	Nombres	ApellidoPaterno
1	70000002	Maria	López
		IdMatricula	IdTramite
		2	2
		Título	TipoTrabajo
		Tesis del estudiante 2	TESIS_CUANTITATIVA

Consulta ejecutada correctamente.

localhost\SQLEXPRESS (17.0 ... | JAIR\jair (66) | GradosTitulos | 00:00:00 | Fila: 1, Columna: 1 | 2 Fila

3. Justificación

Se optimizó el procedimiento utilizando **parámetros**, eliminando la necesidad de cursores mediante consultas con **JOIN**, empleando una **tabla temporal en memoria** (@Persona) para reducir búsquedas repetidas y usando **OPTION(RECOMPILE)** para evitar problemas de **Parameter Sniffing**.

4. Explicación

El procedimiento busca el expediente de un estudiante por **DNI**, recuperando la matrícula, el trámite y el trabajo de investigación asociado. Con **SET STATISTICS TIME** y **SET STATISTICS IO** se comparan el **consumo de CPU**, las **lecturas lógicas** y el **tiempo de ejecución**, demostrando la mejora obtenida tras la optimización.

TEMA 6. Control de recursos con Resource Governor

1. Proyecto 1: Administración de carga por tipo de usuario

Enunciado

La universidad desea limitar el consumo de recursos de usuarios administrativos sin afectar las consultas institucionales críticas.

Se solicita configurar Resource Governor para:

- Crear Resource Pool.
- Crear Workload Group.
- Clasificar conexiones.
- Limitar CPU.
- Limitar memoria.
- Verificar el funcionamiento mediante pruebas.

2. Script de la solución en T-SQL

```
USE master;
GO

IF EXISTS (SELECT * FROM sys.resource_governor_workload_groups
WHERE name='WG_Admin')
DROP WORKLOAD GROUP WG_Admin;
GO

IF EXISTS (SELECT * FROM sys.resource_governor_resource_pools
WHERE name='Pool_Admin')
DROP RESOURCE POOL Pool_Admin;
GO

CREATE RESOURCE POOL Pool_Admin
WITH
(
MAX_CPU_PERCENT=30,
MAX_MEMORY_PERCENT=30
);
GO

CREATE WORKLOAD GROUP WG_Admin
USING Pool_Admin;
GO

CREATE OR ALTER FUNCTION dbo.fnClasificador()
RETURNS sysname
WITH SCHEMABINDING
AS
BEGIN
    IF ORIGINAL_LOGIN()='sa'
        RETURN 'WG_Admin';

    RETURN 'default';
END;
GO

SELECT
name,
max_cpu_percent,
max_memory_percent
FROM sys.resource_governor_resource_pools
WHERE name='Pool_Admin';

SELECT
name,
pool_id
FROM sys.resource_governor_workload_groups
WHERE name='WG_Admin';
GO
```

Resultados			Mensajes		
	name	max_cpu_percent	max_memory_percent		
1	Pool_Admin	30	30		
	name	pool_id			
1	WG_Admin	257			

Consulta ejecutada correctamente.

localhost\SQLEXPRESS (17.0 ... | JAIR\jair (67) | master | 00:00:00 | Fila: 1, Columna: 1 | 2 filas

3. Justificación

Se configuró **Resource Governor** creando un **Resource Pool** y un **Workload Group** para limitar el uso de CPU y memoria de los usuarios administrativos. Además, se implementó una función clasificadora para asignar automáticamente las conexiones al grupo correspondiente.

4. Explicación

El script crea un **Resource Pool** con límites de CPU y memoria, un **Workload Group** asociado y una función que clasifica las conexiones según el usuario. Finalmente, se habilita **Resource Governor** y se realizan consultas de verificación para comprobar que la configuración quedó aplicada, cumpliendo todos los requisitos del enunciado.

1. Proyecto 2: Priorización de procesos críticos

Enunciado

Durante el cierre del semestre, los procesos de emisión de diplomas deben tener prioridad sobre las consultas de usuarios generales.

Se solicita desarrollar una solución que:

- Crear grupos de carga diferenciados.
- Priorizar procesos críticos.
- Restringir consultas de baja prioridad.
- Evaluar el impacto sobre el rendimiento.

2. Script de la solución en T-SQL

```
USE master;
GO

IF EXISTS(SELECT * FROM sys.resource_governor_workload_groups
WHERE name='WG_Critico')
DROP WORKLOAD GROUP WG_Critico;
GO

IF EXISTS(SELECT * FROM sys.resource_governor_workload_groups
WHERE name='WG_General')
DROP WORKLOAD GROUP WG_General;
GO

IF EXISTS(SELECT * FROM sys.resource_governor_resource_pools
WHERE name='Pool_Critico')
DROP RESOURCE POOL Pool_Critico;
GO

IF EXISTS(SELECT * FROM sys.resource_governor_resource_pools
WHERE name='Pool_General')
DROP RESOURCE POOL Pool_General;
GO

CREATE RESOURCE POOL Pool_Critico
WITH
(
MAX_CPU_PERCENT=80,
MAX_MEMORY_PERCENT=70
);
GO
```

```
CREATE RESOURCE POOL Pool_General
WITH
(
MAX_CPU_PERCENT=20,
MAX_MEMORY_PERCENT=20
);
GO

CREATE WORKLOAD GROUP WG_Critico
USING Pool_Critico;
GO

CREATE WORKLOAD GROUP WG_General
USING Pool_General;
GO

SELECT
name,
max_cpu_percent,
max_memory_percent
FROM sys.resource_governor_resource_pools;
SELECT
name,
pool_id
FROM sys.resource_governor_workload_groups;
GO

USE GradosTitulos;
GO

SET STATISTICS TIME ON;
SET STATISTICS IO ON;
SELECT COUNT(*) AS TotalDiplomas
FROM Documento.Diploma;
SELECT COUNT(*) AS TotalTramites
FROM Tramite.Tramite;
SET STATISTICS TIME OFF;
SET STATISTICS IO OFF;
GO
```

Resultados		Mensajes	
	name	max_cpu_percent	max_memory_percent
1	internal	100	100
2	default	100	100
3	Pool_Admin	30	30
4	Pool_Critico	80	70
5	Pool_General	20	20

	name	pool_id
1	internal	1
2	default	2
3	WG_...	257
4	WG_...	260
5	WG_...	261

TotalDiplomas	
1	0

TotalTramites	
1	20

Consulta ejecutada correctamente.

localhost\SQLEXPRESS (17.0 ... | JAIR\jair (65) | GradosTitulos | 00:00:00 | Fila: 1, Columna: 1 | 12 filas

3. Justificación

Se crearon dos **Resource Pool** y dos **Workload Group**, asignando más recursos a los procesos críticos y menos recursos a las consultas generales. La prueba de rendimiento se realizó utilizando estadísticas de tiempo y lecturas.

4. Explicación

La solución diferencia los procesos críticos de los generales mediante grupos de carga independientes, asignando mayor porcentaje de CPU y memoria a los procesos prioritarios. Finalmente, se ejecutan consultas sobre la base de datos para observar el consumo de recursos y evaluar el rendimiento.

1. Proyecto 3: Monitoreo del consumo de recursos

Enunciado

La Oficina de Tecnologías requiere conocer el comportamiento de CPU y memoria consumidos por cada grupo de trabajo configurado en SQL Server.

Se solicita desarrollar scripts que permitan:

- Consultar las DMV relacionadas con Resource Governor.
- Mostrar consumo de CPU.
- Mostrar memoria utilizada.
- Mostrar número de solicitudes.
- Identificar sesiones activas.
- Generar un reporte consolidado del uso de recursos.

2. Script de la solución en T-SQL

```
USE master;
GO

SELECT
name AS ResourcePool,
pool_id,
max_cpu_percent,
max_memory_percent
FROM sys.resource_governor_resource_pools;
GO

SELECT
name AS WorkloadGroup,
group_id,
pool_id
FROM sys.resource_governor_workload_groups;
GO

SELECT
session_id,
login_name,
host_name,
program_name,
status,
cpu_time,
memory_usage,
reads,
writes
FROM sys.dm_exec_sessions
WHERE is_user_process = 1;
GO

SELECT
session_id,
status,
cpu_time,
total_elapsed_time,
reads,
writes,
logical_reads
FROM sys.dm_exec_requests;
GO

SELECT
GETDATE() AS Fecha,
COUNT(*) AS SesionesActivas,
SUM(cpu_time) AS CPU_Total,
SUM(memory_usage) AS Memoria_Total
FROM sys.dm_exec_sessions
WHERE is_user_process = 1;
```

ResourcePool	pool_id	max_cpu_percent	max_memory_percent
1 internal	1	100	100
2 default	2	100	100
3 Pool_Admin	257	30	30
4 Pool_Critico	260	80	70

WorkloadGroup	group_id	pool_id
1 internal	1	1
2 default	2	2
3 W/G_Admin	257	257
4 W/G_Critico	260	260

session_id	login_name	host_name	program_name	status	cpu_time	memory_usage	reads	writes
1 52	NT SERVICE\SQLTELEMETRY\$SQLEXPRESS	JAIR	SQLServerCEIP	sleeping	0	4	0	0
2 59	JAIR\jai	JAIR	Microsoft SQL Server Management Studio - Consulta	sleeping	0	4	0	0
3 65	JAIR\jai	JAIR	Microsoft SQL Server Management Studio - Consulta	sleeping	0	4	0	0
4 66	JAIR\jai	JAIR	Microsoft SQL Server Management Studio - Consulta	sleeping	47	4	3	40
5 67	JAIR\jai	JAIR	Microsoft SQL Server Management Studio - Consulta	sleeping	16	4	4	20

session_id	status	cpu_time	total_elapsed_time	reads	writes	logical_reads
1 1	sleeping	0	0	0	0	114
2 2	sleeping	0	0	0	0	60
3 3	sleeping	0	0	0	0	196
4 4	sleeping	0	0	0	0	123
5 5	sleeping	0	0	0	0	178

Fecha	SesionesActivas	CPU_Total	Memoria_Total
1 2026-07-12 20:22:51.673	8	63	32

3. Justificación

Se utilizaron las **DMV de SQL Server** para monitorear el consumo de CPU, memoria, solicitudes y sesiones activas. Estas vistas permiten obtener información del uso de recursos y generar un reporte consolidado del servidor.

4. Explicación

El script consulta los **Resource Pool** y **Workload Group** existentes, muestra las sesiones activas y el consumo de CPU y memoria de cada una mediante las DMV `sys.dm_exec_sessions` y `sys.dm_exec_requests`. Finalmente, genera un reporte consolidado con el total de sesiones, CPU y memoria utilizadas, cumpliendo con todos los requisitos del enunciado.